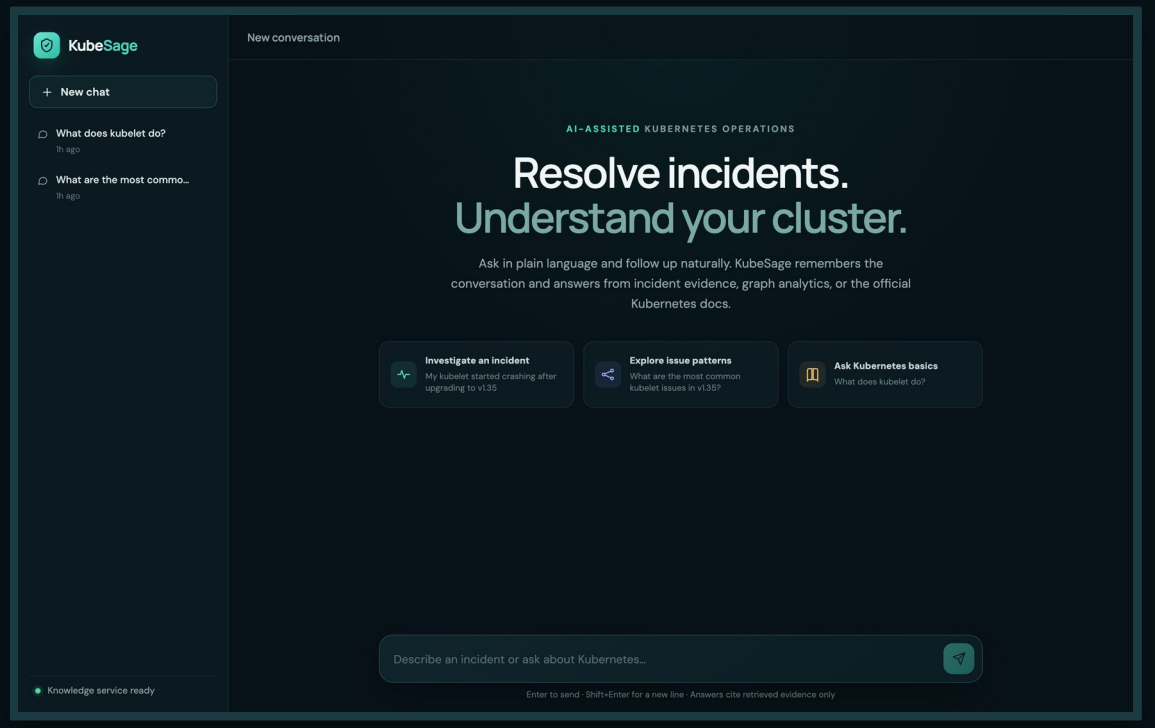
KUBESAGE

Evidence-grounded answers for Kubernetes incidents

A LangGraph and Claude assistant over a Neo4j graph of real Kubernetes issues and the official Kubernetes documentation, running on Azure



One chat, three kinds of evidence

1

Incident troubleshooting

Lists likely causes from the official docs and attaches matching GitHub issues; offers a fix only when one is verified.

“Our GCE e2e job fails: e2e-up.sh exit status 1…”

2

Graph analytics

Counts, rankings and lists over the incident graph through validated, read-only Cypher.

“What are the most common kubelet issues in v1.36?”

3

Kubernetes knowledge

Explains concepts from the official documentation with links to the exact pages used.

“What does the kubelet do?”

4

Conversation memory

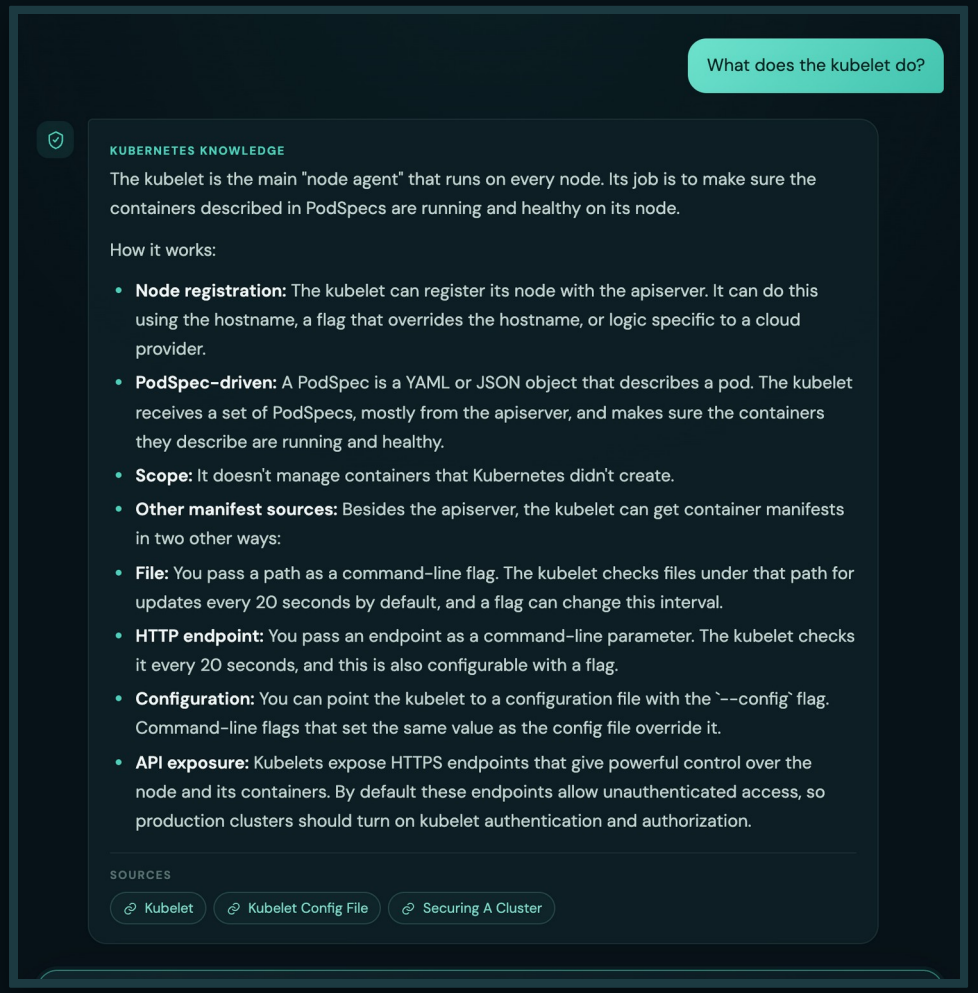
Remembers the chat and a running incident summary; asks one focused question when a report is too vague.

“and how many for the scheduler?”

Examples: what you ask and what you get

You ask	Route	KubeSage answers
What does the kubelet do?	Docs	“The kubelet is the main node agent that runs on every node…” with 3 kubernetes.io sources
Our GCE e2e job fails: e2e-up.sh exit status 1, failed to create master instance	Verified fix	Add --sort-by=~creationTimestamp --limit=1 to the two images list calls; cites Issue #141080 and PR #141082
A new deployment can't download its container image. What are the likely reasons?	Docs-first	Documented causes (wrong image name or tag, missing imagePullSecrets, imagePullPolicy: Never) with kubernetes.io sources, then one narrowing question
Which issues involve both DRA and resourceclaim?	Graph	3 issues: #141293, #140471, #140298, each linked
Kubernetes is not working	Clarify	“That's a frustrating spot to be in. What exactly are you seeing go wrong, and is there an error message?” plus quick replies such as “I'll paste the error message”
…and how many for the scheduler? (after an etcd count)	Memory	Rewritten to “How many issues affect the scheduler component?” → scheduler: 5 issues, kube-scheduler: 2
What's a good recipe for banana bread?	Out of scope	“Baking is outside what I can help with, sorry! I'm here for Kubernetes…” with three example Kubernetes questions to try

Documentation answers cite the pages they use



1

Routed by intent

Claude labels the question “Kubernetes knowledge” and sends it to the docs index.

2

Hybrid retrieval

Vector and BM25 keyword rankings are fused, with at most three chunks per page.

3

Checked citations

Every source chip must be a retrieved kubernetes.io page, or the answer is rejected.

Incidents: the docs explain, GitHub issues add evidence

Docs first. Likely causes and how to check each come from the official docs, ending with one question that narrows it down.

GitHub as evidence. A matching reported failure is attached below the answer; fix steps appear only when a resolution is verified.

A new deployment can't download its container image. What are the likely reasons?

KUBESAGE · FROM THE DOCS

Your new deployment's Pods can't pull their image. The docs point to a few likely causes:

- Invalid image name or tag: the Pod sticks in ImagePullBackOff
- Private registry without imagePullSecrets
- imagePullPolicy: Never, so the kubelet never fetches the image

What status and event message do you see on the failing Pod?

Sources: kubernetes.io · Images, Deployments

Our control plane keeps dying on v1.29.15: etcd exits with exitCode 0, reason Completed

KUBESAGE · WITH GITHUB EVIDENCE

That pattern has been reported on v1.29.15: etcd starts, becomes leader, then exits with exitCode 0, taking kube-apiserver down with it. No confirmed fix is known yet. Do the controller-manager and scheduler drop at the same moment?

Similar GitHub reports (no verified fix yet)

- #140743 · etcd static pod exits after startup while the kubelet runs

Sources: github.com/kubernetes/kubernetes/issues/140743

Replies that talk like a colleague, checked like code

1 One focused question

A vague report gets one natural question for the detail that matters most, plus quick-reply buttons.

2 Docs first

Likely causes come from the official docs; matching issues and verified fix steps are attached verbatim.

3 Grounded suggestions

Up to three follow-ups, each naming a component, version, error or doc topic found in the evidence.

4 Every detail checked

Issue numbers, versions, counts and commands in a reply must appear in the evidence, or it is retried.

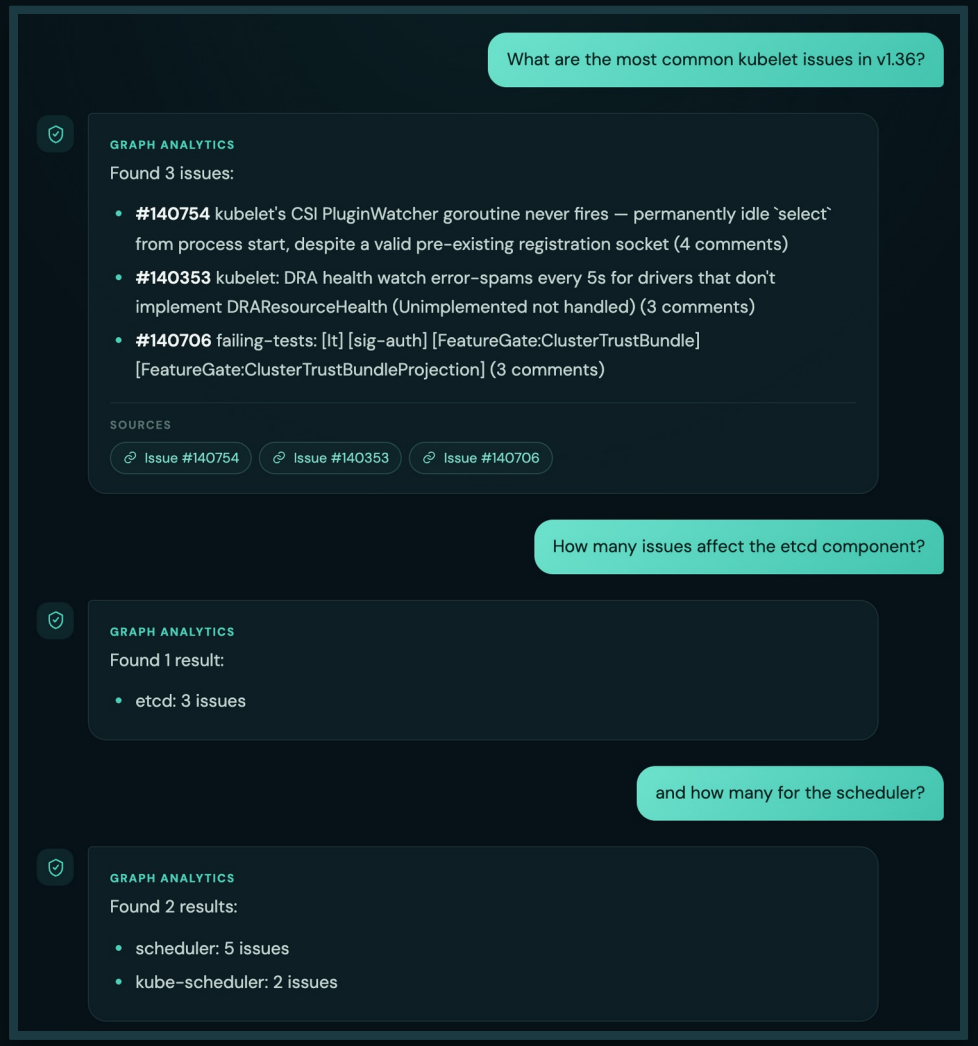
5 Off-topic declined

An off-topic, destructive or injected answer to a pending question is declined, never merged.

6 Incident summary

Version, exact errors, ruled-out causes and the open question are kept for the whole chat.

Analytics answers come straight from the graph



1

Template queries

Counts, top-N, lists and multi-component questions map to fixed Cypher templates; Claude only extracts the filters.

2

No model in the answer

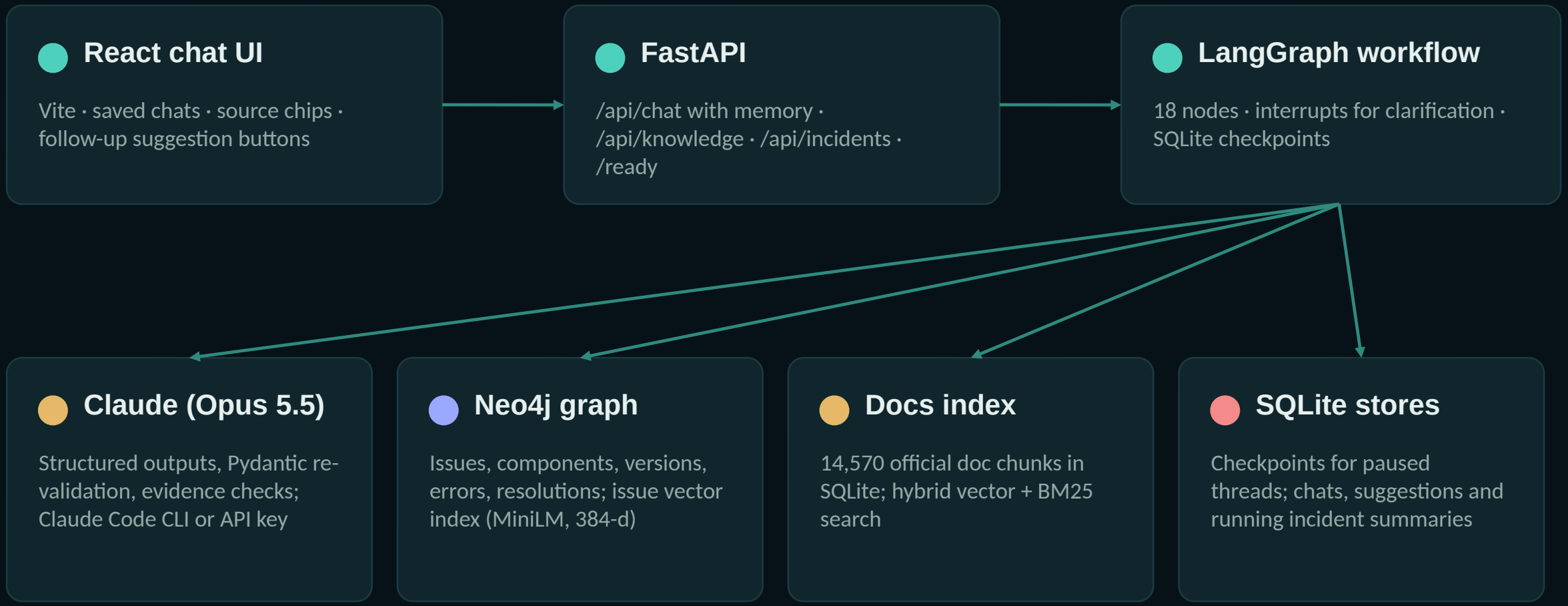
Rows are rendered directly, with issue links as citations and a note when a list is cut short.

3

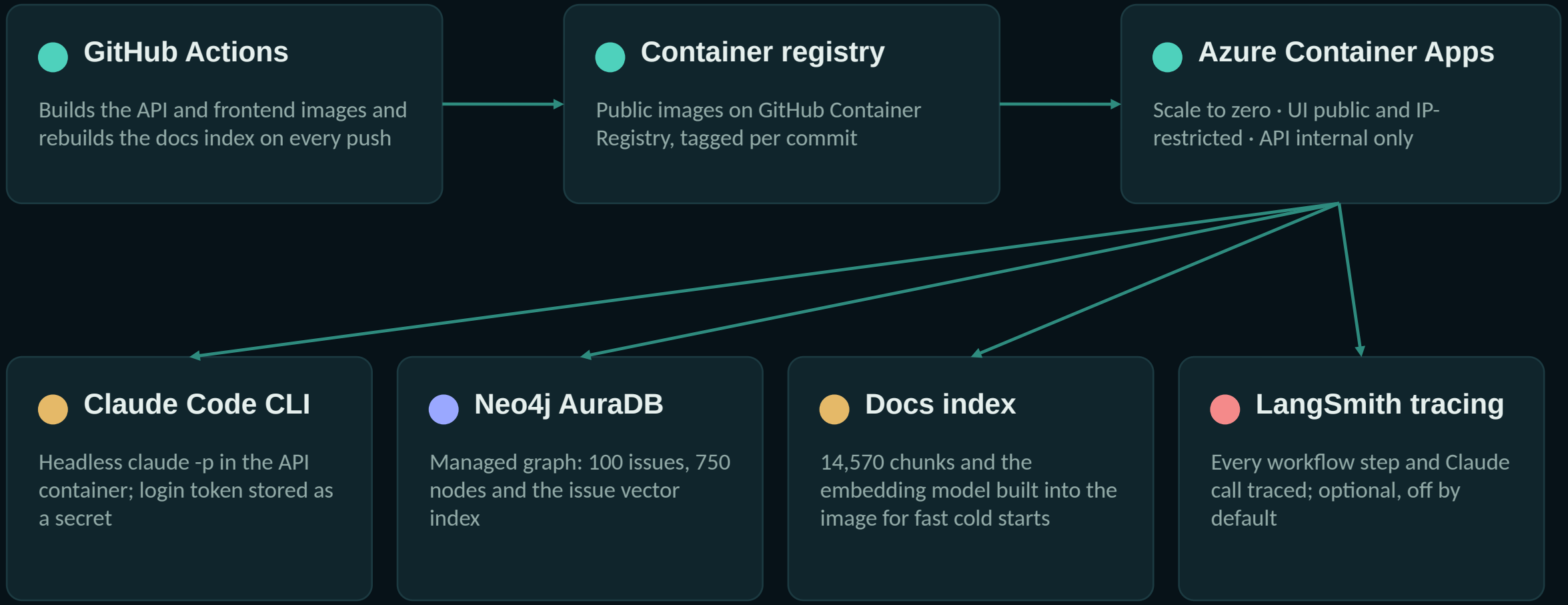
Memory chain

“and how many for the scheduler?” is rewritten by Claude from the last six messages into a standalone question.

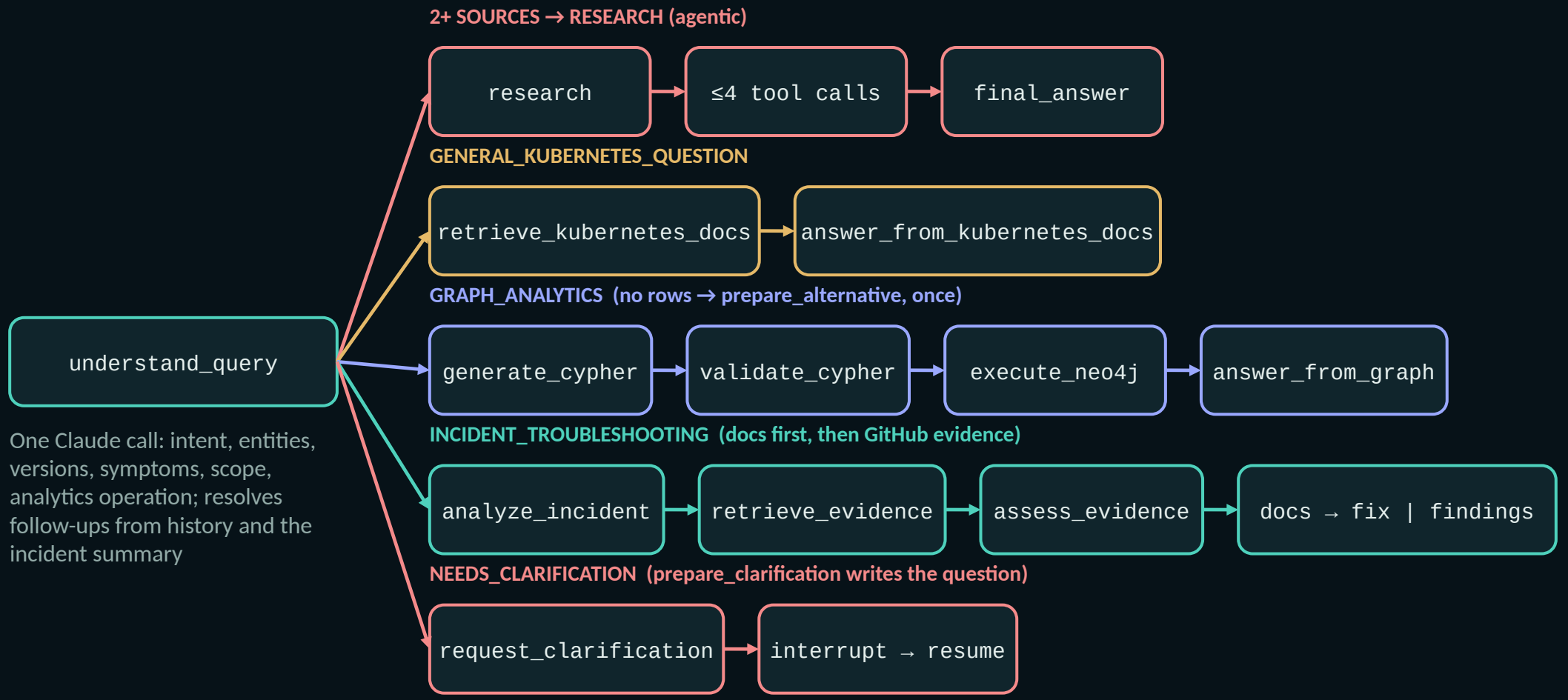
Architecture



Deployment: Azure Container Apps



LangGraph workflow: one graph, five routes



One Claude call: intent, entities, versions, symptoms, scope, analytics operation; resolves follow-ups from history and the incident summary

Invalid Cypher is regenerated once; an empty result gets one broader query. A failed research loop falls back to the primary route. Off-topic answers to a pending question are declined; clarification answers loop back (max three rounds).

Flow: documentation questions



Retrieval

Three rankings fused with reciprocal rank fusion; API-reference pages ranked lower; at most 3 chunks per page.

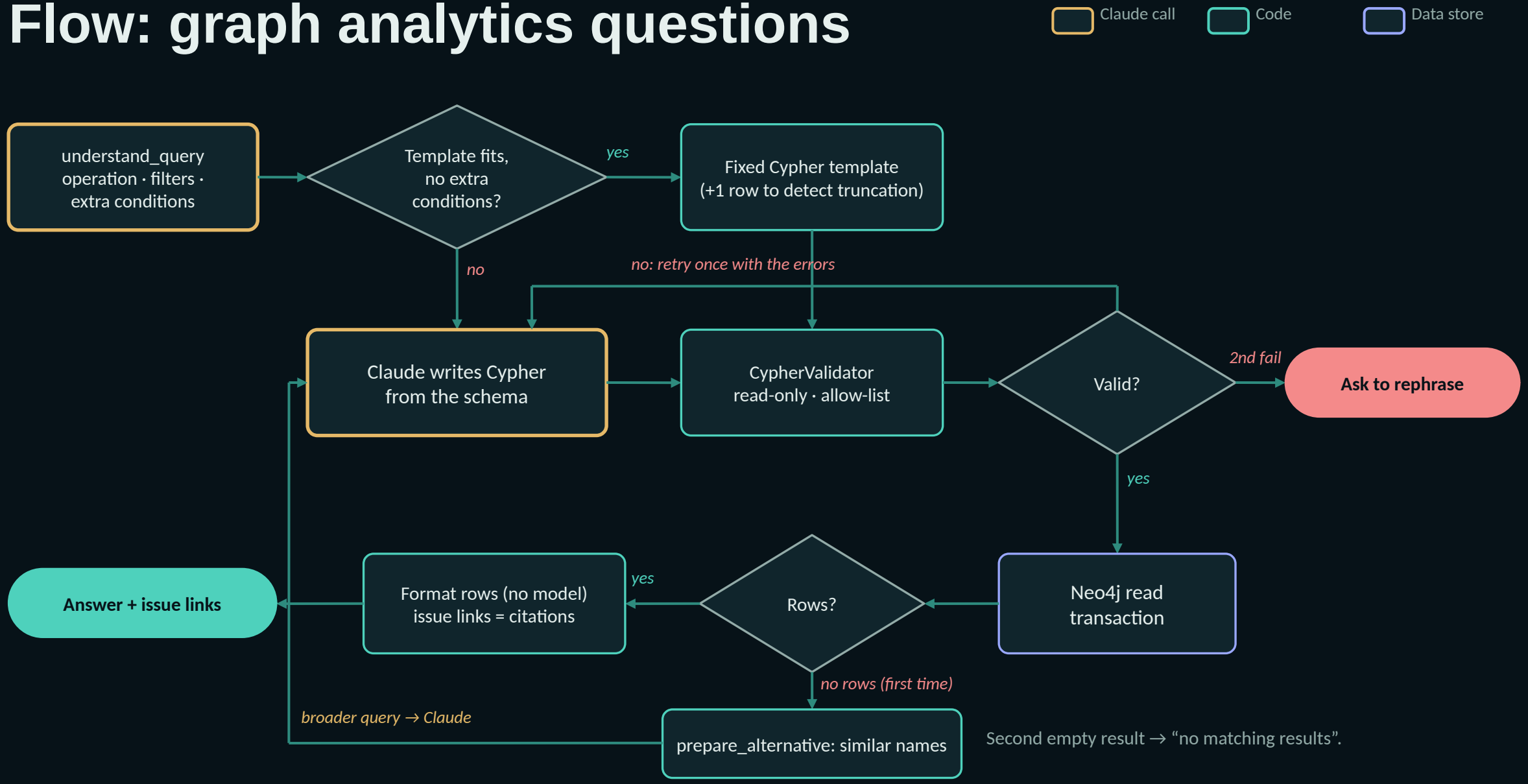
Grounding

The prompt forbids facts from general knowledge; every citation must be the URL of a retrieved chunk, or the reply is retried once with the error.

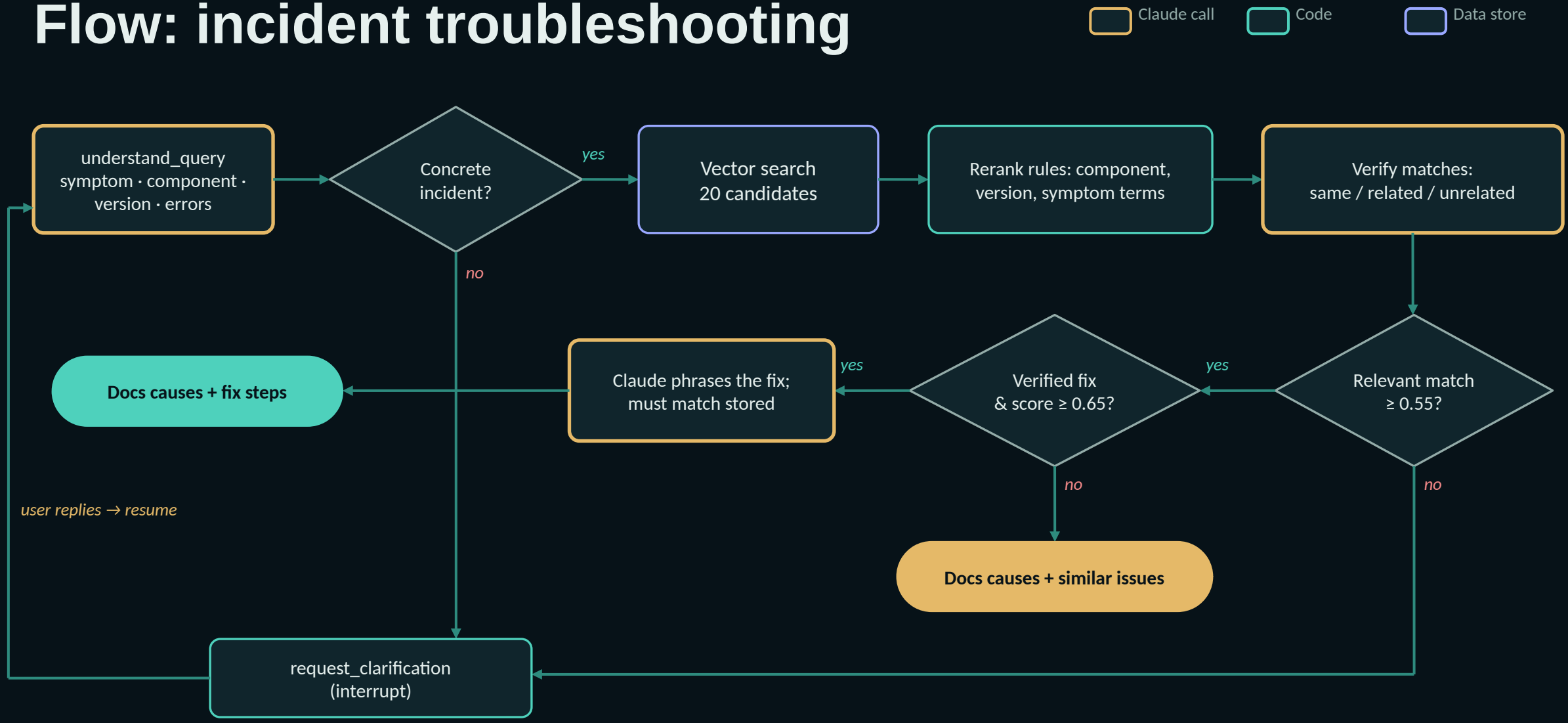
Fail-safe

If Claude is unavailable or keeps citing other pages, the user still gets an excerpt of the best documentation chunk.

Flow: graph analytics questions

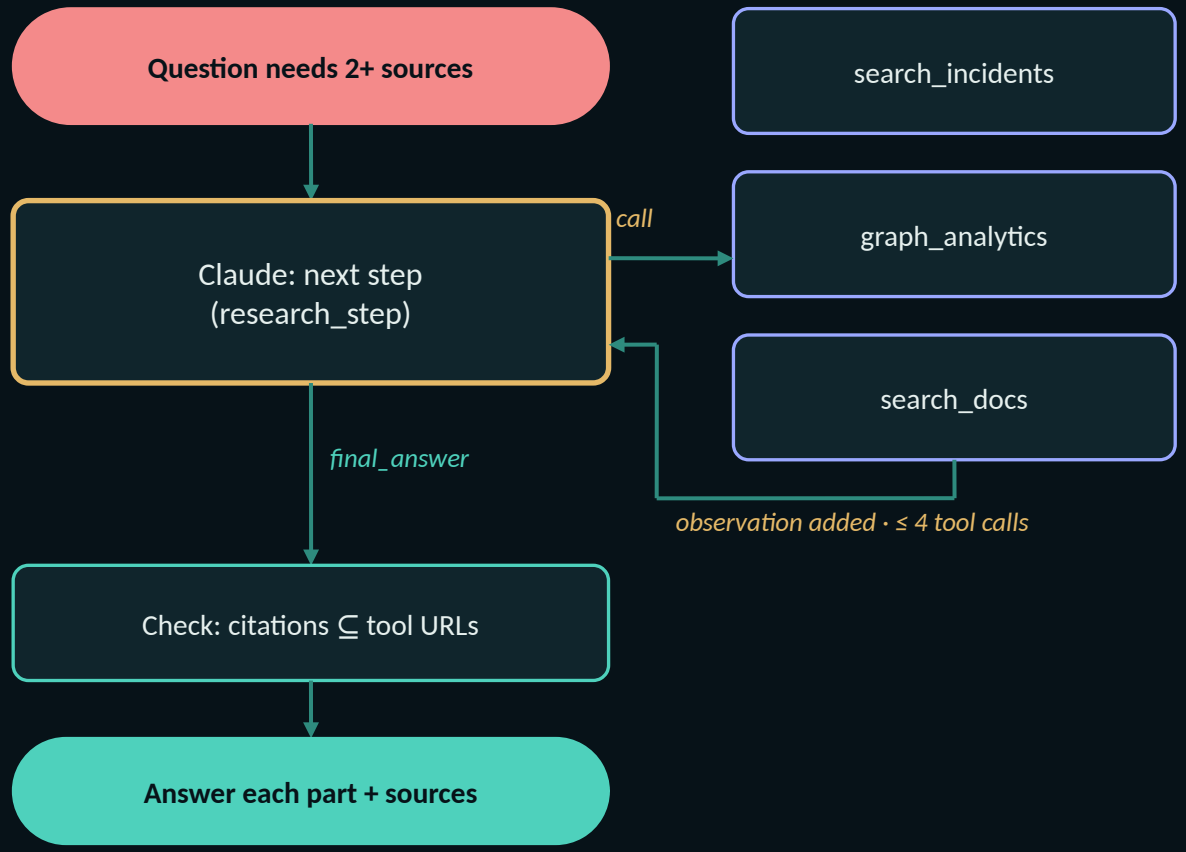


Flow: incident troubleshooting



Each outcome first goes to troubleshoot_from_docs: the docs explain the causes and this evidence is attached. Max three clarification rounds.

Agentic research for multi-source questions



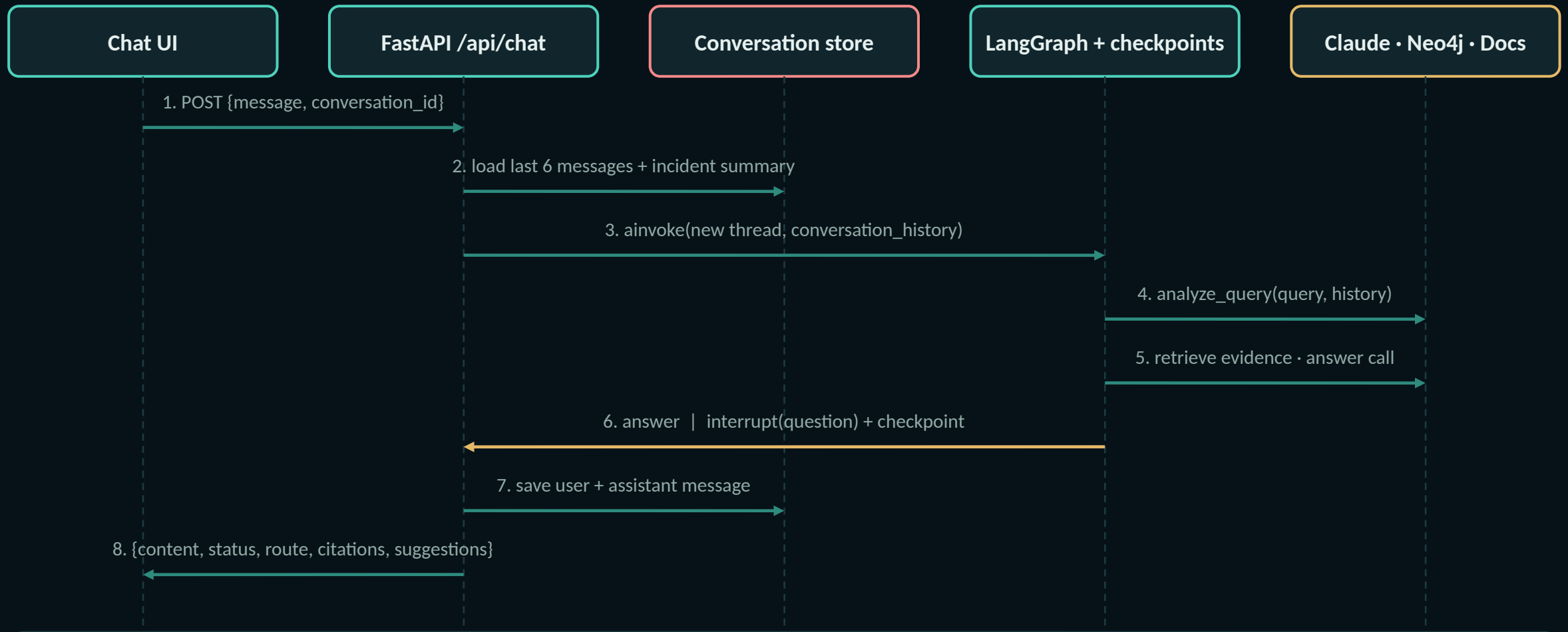
Failure → the question's normal route

Real trace: "How many issues affect etcd, and what does etcd do according to the documentation?"

#	Claude chose	Why
1	`graph_analytics (COUNT etcd)`	Need the count of reported issues affecting the etcd component.
2	`search_docs "etcd key value store Kubernetes cluster data"`	Count is known (3 issues); still need the official docs description of what etcd does.
3	`final_answer`	The observations already answer both parts: the issue count and what the docs say etcd does.

Answer: 3 reported issues affect etcd · etcd is "a consistent and highly-available key value store used as Kubernetes' backing store for all cluster data" — citing the etcd glossary and the operating-etcd task page.

A chat message, end to end



Clarification: if the last reply asked for details, step 3 resumes the paused thread instead. After each reply, a background task updates the chat's running incident summary.

Memory chain: follow-ups become standalone questions

Conversation (SQLite)

You: How many issues affect the etcd component?

KubeSage: Issue count: 3

You: and how many for the scheduler?

last 6 messages

Claude

analyze_query(query, history)

```
{
  "normalized_query": "How many issues affect
the scheduler component?",
  "intent": "GRAPH_ANALYTICS",
  "components": [
    "scheduler"
  ],
  "analytics_operation": "COUNT"
}
```

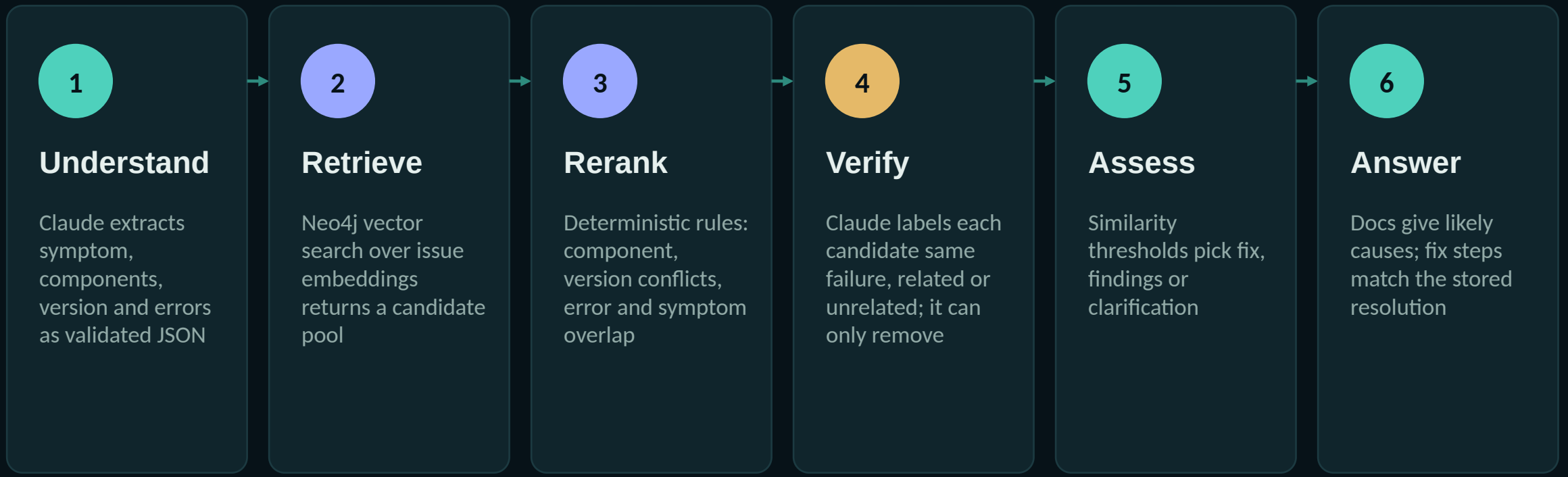
Why it stays grounded

History and the incident summary only resolve references and fill in earlier details. Retrieval and answers use the standalone question; a new topic inherits nothing.

Graph template answer

scheduler: 5 issues · kube-scheduler: 2 issues

Inside an incident question



Why step 4 exists: rules alone accept issues that only share a component and version. Verification keeps those out, so only the same failure is shown as a match.

Worked example: tracing an incident question

```
“The kube-controller-manager garbage collector
keeps reprocessing objects that have a valid
owner several times per second on Kubernetes
v1.36.2”
```

```
{
 "components": [
  "kube-controller-manager",
  "garbage collector"
 ],
 "kubernetes_version": "v1.36.2",
 "has_sufficient_details": true,
 "missing_details": [
  "exact error message or error code",
  "relevant logs"
 ]
}
```

① Claude extracts structured facts

Issue	Vector	Final	Decision	Why
#141382	0.92	0.92	**Accepted**	component + symptom terms: objects, owner, valid
#140561	0.80	0.72	**Rejected**	only shares kube-controller-manager
#140471	0.78	0.51	**Rejected**	different component; no matching symptom terms; score too low
#140242	0.76	0.59	**Rejected**	different component; no matching symptom terms
#141293	0.76	0.54	**Rejected**	different component; no matching symptom terms; score too low
#140743	0.75	0.59	**Rejected**	only shares kube-controller-manager; conflicting version

② Neo4j vector search, then deterministic reranking (top 6 of 20 candidates)

③ Result: findings for #141382, no fix offered

#140561 shares only the kube-controller-manager component, so the reranker rejects it. No verified resolution exists, so no fix is offered.

Worked example: verification removes look-alikes

"A user with only namespace patch permission labelled their namespace with the DRA admin-access label and a ResourceClaim with adminAccess: true was then accepted"

Rules accept 5

All five share "DRA" words and the dynamic-resource-allocation component

Issue	Claude label	Outcome
#141293	same_failure	kept as the match
#140298	unrelated	dropped
#140441	unrelated	dropped
#140457	unrelated	dropped
#140472	unrelated	dropped

Answer

Findings: #141293 — Non-admin users can self-escalate to DRA adminAccess by labelling their namespace. No verified fix is offered.

Safety rules for this step

Claude can only remove or demote candidates, never add one. Issue numbers must match the supplied set exactly. If the call fails, the rule-based matches are kept.

Worked example: from question to Cypher to answer

1. Question

“Which issues involve both DRA and resourceclaim?”

```
{"intent":"GRAPH_ANALYTICS",
"analytics_operation":"RELATIONSHIP",
"components":["DRA",
"resourceclaim"]}
```

2. Claude: structured operation

```
MATCH (i:Issue)-[:AFFECTS_COMPONENT]->(c:Component)
WHERE replace(toLower(c.name), 'kube-', '') IN $components
WITH i, count(DISTINCT replace(toLower(c.name), 'kube-', '')) AS
matched_components
WHERE matched_components = $component_count
RETURN i.issue_number AS issue_number, i.title AS title, i.html_url AS url
ORDER BY issue_number DESC LIMIT 21
// $components = ["dra","resourceclaim"]
```

3. Fixed template (no model-written Cypher); LIMIT 21 = 20 shown + 1 to detect truncation

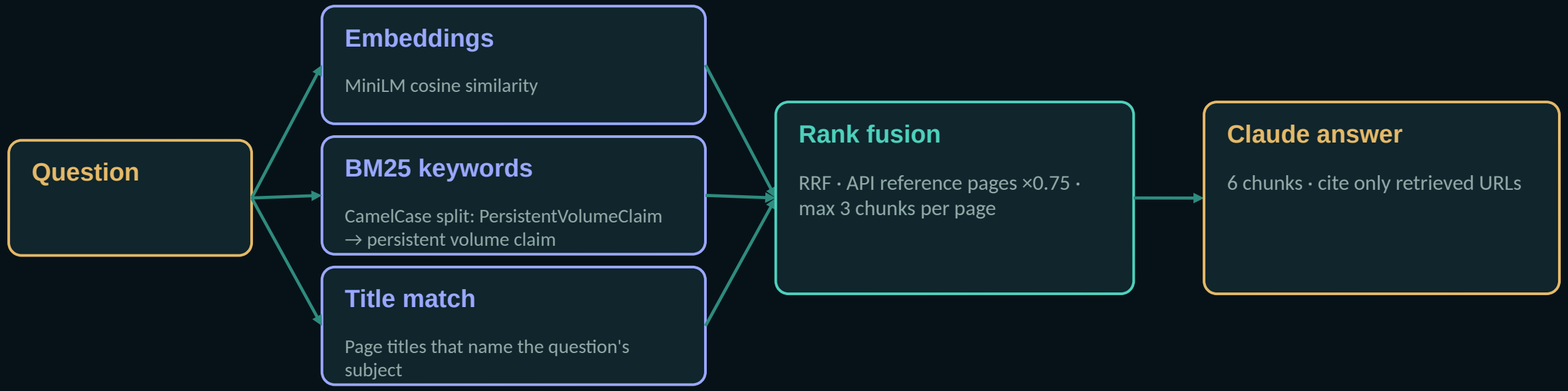
4. CypherValidator

- ✓ read-only, no procedures
- ✓ known labels and relationships
- ✓ values as $parameters
- ✓ LIMIT ≤ 100

5. Answer from rows

Found 3 issues:
#141293 Non-admin users can…
#140471 DRA: re-preparing a claim…
#140298 [sig-scheduling]…

Hybrid documentation search



Example: "How do Deployments perform rolling updates?"

Top 3 results

1. Deployments — Writing a Deployment Spec › Strategy ›…
2. Deployments — Use Case
3. Performing a Rolling Update — Updating an application

Grounding and safety guardrails

1 Schema-constrained output

Every Claude reply follows a JSON schema, is re-validated with Pydantic, and is retried once with the error.

2 Evidence checks

Fix steps and citations must match supplied evidence; every number, version and command in a reply must appear in it.

3 Read-only Cypher

Allow-listed labels and relationships, no writes or procedures, parameters only, LIMIT ≤ 100, read transactions.

4 Untrusted input

User, issue and doc text are data, never instructions; a prompt-injected DETACH DELETE leaves the graph unchanged.

5 Out-of-scope replies

Off-topic or destructive requests, including answers to a pending question, get a short scope reply.

6 Safe failure

If Claude is down, routing reports “temporarily unavailable”; answer steps fall back to stored evidence.

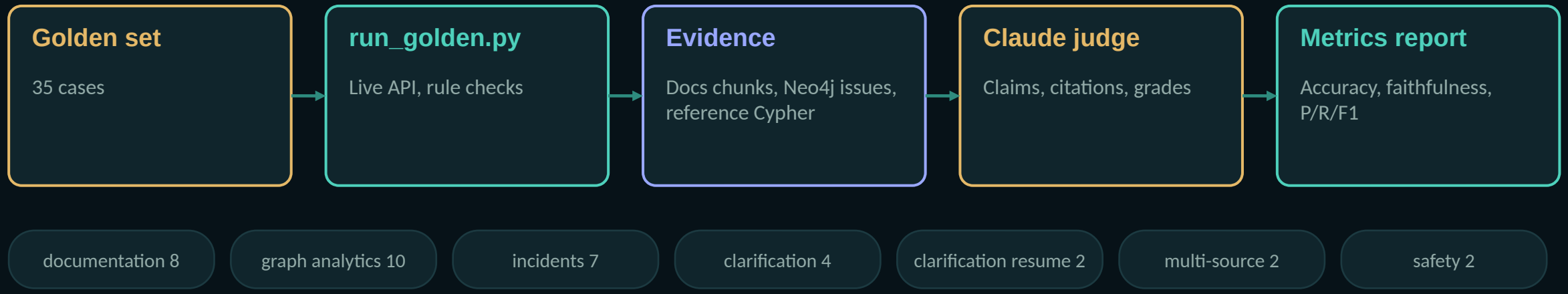
Offline data pipeline



Ingest	Enrich	Evidence	Resolve	Graph	Embed	Docs
100 closed Kubernetes issues from the GitHub API	Claude extracts summary, symptom, components, errors, versions	Comments, linked PRs and closing commits per issue	Claude extracts a verified resolution; URLs must be supplied	Neo4j load with MERGE only; never deletes data	MiniLM embeddings and a Neo4j vector index	Official docs fetched, chunked and indexed in SQLite

Every stage is resumable and idempotent. A closed issue is never treated as fixed without evidence: only fixed or configuration outcomes with verified steps become actionable fixes.

How we measure quality



Example claim check (doc-04)

Answer	“The kubelet --resolv-conf flag can be set to "" to stop inheritance…”
Judge label	UNSUPPORTED — not present in the provided evidence
Counts toward	claim hallucination rate and that answer's faithfulness

Deterministic metrics (no judge)

Intent and ask-vs-answer P/R/F1 · citation validity · expected-source recall · incident and graph precision/recall against reference queries

Golden-set quality: latest judged run



83%

fully correct

25/30

judged correct

6%

of claims unsupported

Results by category

Category	Cases	Correct	Unsupported claims
documentation	8	5/8	6
graph analytics	8	7/8	3
incident fix	2	2/2	0
incident findings	4	4/4	0
clarification	4	4/4	0
clarification resume	2	1/2	2
safety	2	2/2	0

Routing and decisions

Intent accuracy	100%
Intent macro F1	1.00
Ask-vs-answer accuracy	96%
Citation validity	100%
Expected-source recall	88%
Incident recall	88%
Graph recall	100%

Where it stands, and what is next

Strengths today

Docs-first troubleshooting

Likely causes from the official docs, with matching GitHub issues and verified fixes attached as evidence.

Grounded conversation

One focused question, evidence-backed suggestions and a running incident summary; every detail is checked.

Read-only, self-correcting analytics

Fixed Cypher templates, validated model-written queries and one broader retry when nothing matches.

Cloud-ready and observable

Azure Container Apps with scale to zero, AuraDB, a private API and LangSmith tracing.

Next steps

Docs completeness

Some answers miss the core point the docs contain: the HPA replica formula, StatefulSet identity and storage. Next: section-aware chunking.

Run-to-run variation

Claude names components differently between runs, which moved one case to clarification. Next: paraphrase variants and repeated runs.

Evaluation scale

35 golden cases judged by the same model family. Next: re-judge the conversational layer, a larger set and a second judge.